

Assignment 2: Data Modelling

Danial Ansari (s4119075); COSC2670

Before starting the tasks I checked the data. All 13,611 rows are complete (no missing values) and every measurement is positive, but 68 rows are exact duplicates. I removed those so the same bean could not end up in both a training and a test set, which left 13,543 rows. From these I drew the three random samples (300, 2,000 and 2,000 rows) with a fixed seed and saved them as COSC2670-s4119075-A2SampleOne/Two/Three.csv. Each task loads its own saved file, so all results within a task come from the same sample.

Task 1: Regression

1.1 Relationship between MajorAxisLength and Perimeter

Figure 1 shows that the two variables have a very strong, positive and almost straight-line relationship (Pearson $r = 0.977$). This makes sense, because a longer bean has a longer outline. I coloured the points by bean class only to help explain what I saw. Three things stood out to me.

First, the 13 Bombay beans sit on their own in the top-right corner (lengths of 526–680 px). They are the only outliers on both variables, but they are real, large beans that follow the same trend, so I kept them. Between about 450 and 525 px there are no beans at all, so the data are not spread evenly, and this small group has a lot of pull on the fitted line.

Second, the classes form slightly different bands. At the same length, rounder beans (Seker, Barbunya) have a longer perimeter than elongated ones (Horoz, Cali). So perimeter depends on shape as well as on length.

Third, the spread gets wider as beans get bigger, and it is widest for Barbunya. One Barbunya bean of about 405 px has a perimeter of 1,340 px, well above the others, which points to an irregular outline.

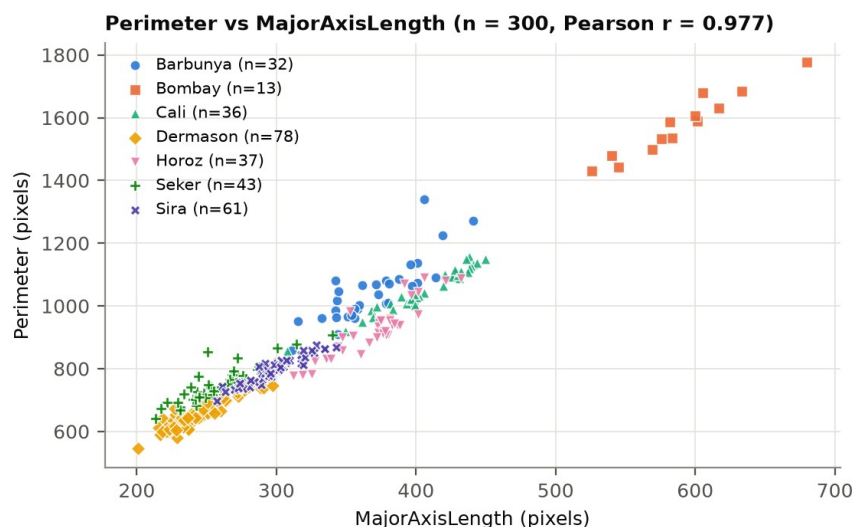


Figure 1. Perimeter against MajorAxisLength for the 300 sampled beans, coloured by class.

1.2 Simple linear regression model

I split the sample randomly into 240 training and 60 test beans and fitted an ordinary least-squares line on the training beans, with Perimeter as the dependent variable:

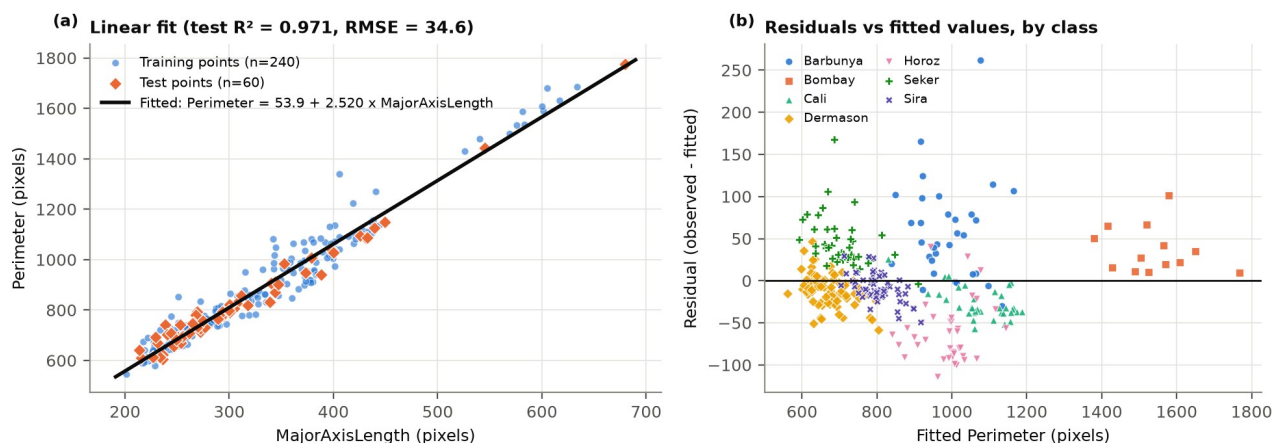
$$\text{Perimeter} = 53.90 + 2.520 \times \text{MajorAxisLength}$$

The slope means that each extra pixel of length adds about 2.52 pixels of perimeter on average (95% confidence interval 2.45 to 2.59). This value seemed reasonable to me: a very flat ellipse has a perimeter close to $2 \times$ its length, and a circle has $\pi \times$ its diameter $\approx 3.14 \times$, so a bean-shaped object should land somewhere between the two. The intercept (53.9) would be the perimeter of a bean with zero length. That is far outside the data (the shortest bean is 201 px), so it has no real meaning and just positions the line. It is probably positive because perimeter also depends on width, which this model leaves out.

Table 1. Regression performance (errors in pixels).

Data	R ²	RMSE	MAE	MAPE
Training set (n = 240)	0.951	48.95	35.35	3.98%
Test set (n = 60)	0.971	34.60	27.29	3.40%
10-fold CV, all 300 (mean ± SD)	0.953 ± 0.019	45.11 ± 12.68	—	—

Overall the line represents the relationship well. It explains about 95–97% of the variation in perimeter, and the typical error is around 30–35 px, or 3–4% of a bean's perimeter (Table 1). The test score is a little higher than the training score, but that is just luck of the split: only 3 of the 60 test beans are Barbunya, the class with the largest errors. Cross-validation gives a steady R² of 0.953. The residual plot (Figure 2b) shows the main weakness. The errors are not random but depend on the class. Horoz beans are over-predicted by about 61 px on average, while Seker, Barbunya and Bombay are under-predicted by 37–59 px. The residuals are strongly related to aspect ratio ($r = -0.72$), so the missing piece is shape. Adding a width or shape variable would fix most of this.

**Figure 2.** (a) Training and test beans with the fitted line; (b) residuals against fitted values, coloured by class.

Task 2: Classification

Sample Two has 2,000 beans, and the classes are uneven: from 546 Dermason down to only 80 Bombay. I made one stratified 80/20 split (1,600 training and 400 test beans), so both sets keep the same class proportions, and used this same split for every model. All tuning was done on the training set with 10-fold stratified cross-validation repeated 3 times. Scaling was placed inside a pipeline so it was only ever fitted on training folds. Because of the class imbalance I used macro F1 (every class counts equally) to choose settings. For the final test I report accuracy, macro precision, macro recall and macro F1.

2.1 k-Nearest Neighbours (kNN)

kNN decides by distance, so the feature scales matter a lot. Area is in the tens of thousands while ShapeFactor2 is around 0.002, so without scaling Area would decide almost everything. I therefore standardised all 16 features. The difference was large: the best cross-validated macro F1 was only 0.631 without scaling, compared with 0.931 with it.

To choose k , I tried every value from 1 to 40 (Figure 3a). Very small k did poorly, because the model reacts to single noisy beans ($k = 1$ gave 0.907 and $k = 2$ gave 0.894). From about $k = 8$ to 20 the score is flat, and after that it slowly drops as the neighbourhoods get too big and smaller classes get out-voted. I chose $k = 14$ because it had the best score (macro F1 0.931) and sits in the middle of the flat region, so a slightly different k would give almost the same result.

On the test set the tuned kNN reached an accuracy of 0.923, macro precision 0.931, macro recall 0.927 and macro F1 0.929. Bombay was classified perfectly, since it is so much bigger than the other beans. Most of the 31 mistakes were between Dermason and Sira (13) and between Barbunya and Cali (7), as Figure 3b shows. These pairs simply look alike.

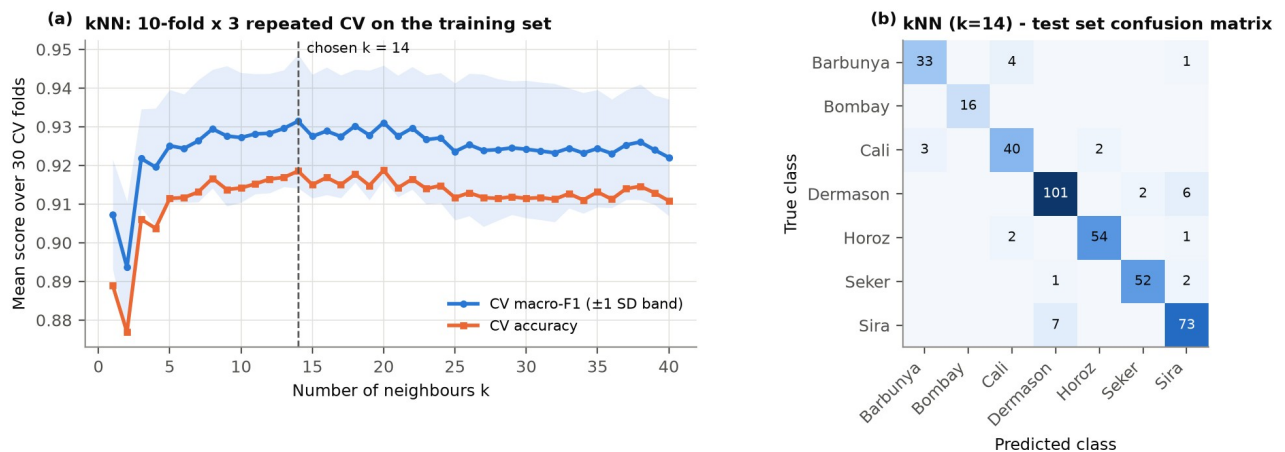


Figure 3. (a) Cross-validated macro F1 and accuracy for $k = 1\text{--}40$; (b) test confusion matrix for $k = 14$.

2.2B Advanced improvement strategy: Local Mean-based kNN

Looking at the kNN errors, I thought the majority vote itself was part of the problem. Near the Dermason/Sira border most neighbours tend to be Dermason simply because there are more of them (546 vs 400). One odd neighbour also counts as a full vote, and the vote ignores how close each neighbour actually is.

I therefore changed how the neighbours are used, following the local mean-based kNN idea of Mitani and Hamamoto (2006). I wrote the classifier myself as a scikit-learn style class. For a new bean x :

- for each class separately, it finds the k training beans of that class that are closest to x ;
- it averages those k beans into one 'local mean' point for that class;
- it predicts the class whose local mean is closest to x .

This changes three things. Every class gets exactly k representatives, so a big class cannot win just by being big. Averaging smooths out single noisy beans. And the decision is a distance, not a vote, so there are no ties. With $k = 1$ it behaves exactly like 1-NN, and with very large k it becomes a 'closest class centre' rule. I also tried a weighted version that combines local means of 1, 2, ..., k neighbours (LMPNN, Gou et al., 2014). Both versions used the same pipeline, the same folds, the same k range and the same test split as the original kNN.

Table 2. Classification results (CV = repeated cross-validation on the training set; test = 400 beans).

Model	Chosen parameter(s)	CV macro-F1	Test acc.	Test macro-P	Test macro-R	Test macro-F1
kNN (original, 2.1)	$k = 14$	0.931	0.923	0.931	0.927	0.929
Local mean kNN (2.2B)	$k = 9$	0.936	0.935	0.941	0.935	0.938
Weighted version (LMPNN)	$k = 38$	0.934	0.928	0.937	0.933	0.935
Decision Tree (2.3)	entropy, depth 6, leaf 10	0.907	0.873	0.880	0.879	0.879

The simple local mean version had the best cross-validation score ($k = 9$), so that is my proposed model. It beat the original kNN on every test metric: accuracy went from 0.923 to 0.935 (26 errors instead of 31) and macro F1 from 0.929 to 0.938 (Table 2, Figure 4b). The gains are where I expected them, in the similar-looking small beans. Dermason F1 went from 0.927 to 0.950, Sira from 0.896 to 0.915 and Seker from 0.954 to 0.972 (Figure 4c). Figure 4a also shows that the original kNN gets worse as k grows, while both local mean versions stay higher and are less affected by the choice of k .

I would call this a real but small improvement. It shows up in cross-validation and on the test set. However, only 9 test beans are right for one model and wrong for the other (7 in favour of the new method, 2 against), and a McNemar test gives $p = 0.18$, so 400 test beans are not enough to prove it statistically. It helps when classes overlap and differ in size, but it did nothing for Barbunya and Cali (F1 unchanged), because those two really do overlap in this feature space (Figure 8a), and no distance-based rule can separate beans that sit on top of each other. If k is too large, the local mean also gets pulled across the class border, which is why the best k is smaller than for normal kNN. The extra cost is small: predicting the 400 test beans took 14 ms instead of 9 ms.

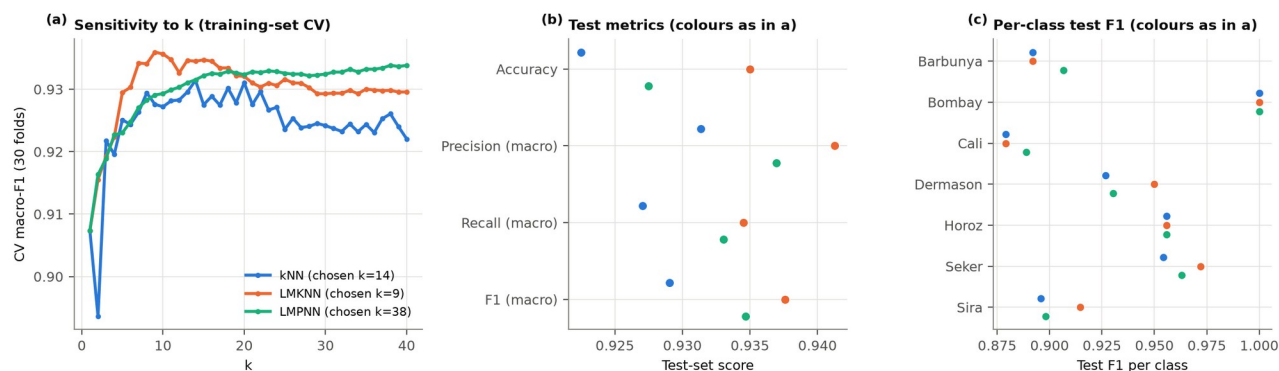


Figure 4. (a) Cross-validated macro F1 against k for the original kNN and the two local mean versions; (b) test metrics on the same split; (c) test F1 for each class.

2.3 Decision Tree

A decision tree splits on one feature at a time, so it does not need scaling. I tuned the tree depth (1–20 or unlimited), the minimum number of beans per leaf (1, 2, 5, 10 or 20) and the split criterion (Gini or entropy) together, using the same cross-validation. The best combination was **entropy, depth 6 and at least 10 beans per leaf** (CV macro F1 0.907), which gives a tree with 38 leaves. The validation curve in Figure 5a explains why I settled on depth 6. Shallower trees underfit (depth 3 only reaches 0.75), the validation score peaks at 6, and deeper trees only improve the training score. A fully grown tree scores 1.000 on training data but only 0.895 in cross-validation, which is clear overfitting. The 10-bean minimum stops leaves from being built around single beans.

On the test set the tree got an accuracy of 0.873 and a macro F1 of 0.879, about 5 points below kNN on every metric (Table 2, Figure 5b). Its cross-validation score was also lower (0.907 vs 0.931), so this was not just a bad split. The tree struggled most with Barbunya (F1 0.73, with 9 of 38 called Cali), Sira (0.82) and Cali (0.83). Figure 5c shows that it relies mostly on size (MajorAxisLength, MinorAxisLength) and then on shape (ShapeFactor1 and 3). Duplicate size measures such as ConvexArea were not used at all.

Comparing the two models on this dataset:

- **Predictive performance:** kNN is clearly better. The borders between bean types are smooth diagonal mixes of size and shape, which kNN can follow. A tree can only make straight cuts on one feature at a time, so it needs many steps to follow a diagonal border.
- **Interpretability:** the tree wins. I can read its rules and see which features matter (for example, Bombay is separated by a single size cut). kNN cannot give a reason beyond 'these beans look similar'.
- **Complexity:** kNN has almost no training step, but it must keep all 1,600 training beans and compare every new bean with all of them. The tree takes a little longer to train (23 ms vs 7 ms) but is a small model that predicts faster (2.5 ms vs 5.5 ms).
- **Sensitivity to the data:** kNN depends heavily on scaling (0.63 vs 0.93), on duplicated features and on k . The tree does not care about scaling and ignores useless features, but small changes in the data can change its splits, and it overfits if depth is not limited.

If accuracy is the goal I would use kNN (or the local mean version). If the result has to be explained to someone, the tree is the better choice.

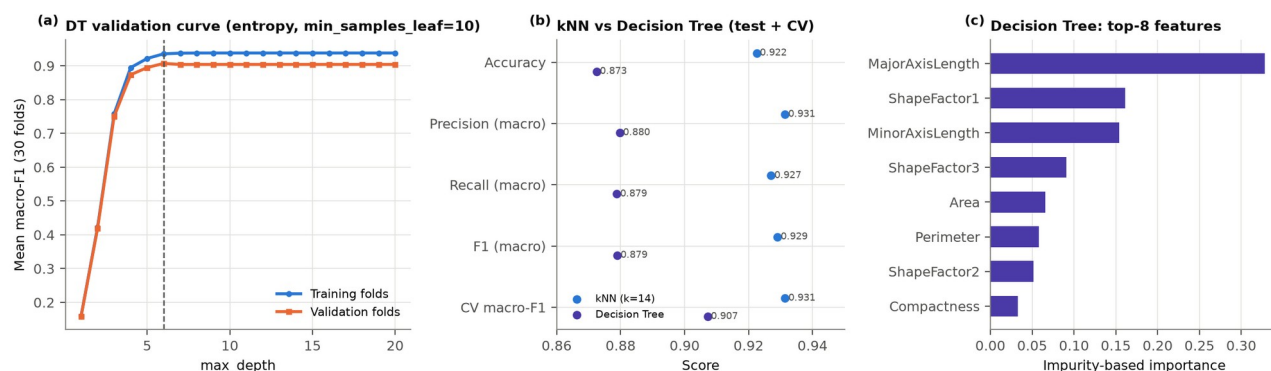


Figure 5. (a) Training and validation macro F1 against tree depth; (b) kNN and the decision tree on the same metrics; (c) the eight most important tree features.

Task 3: Clustering

For this task I used Sample Three (2,000 beans) and all 16 features. I removed the Class column before clustering and did not use it to choose any settings. It was only used afterwards to check how well the clusters matched the real bean types, using the adjusted Rand index (ARI, 0 = random, 1 = perfect) and normalised mutual information (NMI). All features were standardised first. To measure cluster quality without labels I used the silhouette score (higher is better), the Davies–Bouldin index (lower is better) and the Calinski–Harabasz index (higher is better).

3.1 k-Means

I ran k-Means for $k = 2$ to 12 (Figure 6). The measures did not all agree. The elbow of the inertia curve is around $k = 5$, but it is not a sharp bend. The silhouette is highest at $k = 3$ (0.403), Davies–Bouldin is lowest at $k = 3$ (0.910), and Calinski–Harabasz prefers $k = 2$. Since two of the measures clearly point to it, I chose **$k = 3$** .

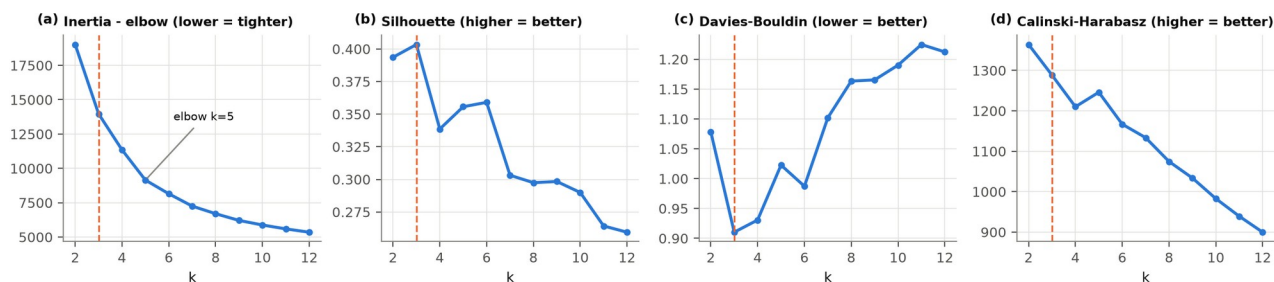


Figure 6. How k affects (a) inertia, (b) silhouette, (c) Davies–Bouldin and (d) Calinski–Harabasz. The dashed line marks $k = 3$.

Table 3. The three k-Means clusters compared with the real classes, plus average size and shape.

Cluster	Barbunya	Bombay	Cali	Dermason	Horoz	Seker	Sira	Mean Area	Aspect ratio
0 ($n = 820$)	190	0	247	5	274	0	104	64,036	1.78
1 ($n = 1,098$)	17	0	0	525	4	295	257	37,237	1.43
2 ($n = 82$)	1	81	0	0	0	0	0	172,026	1.59

With $k = 3$ the silhouette is 0.403, Davies–Bouldin 0.910 and Calinski–Harabasz 1,288. Against the real classes the ARI is 0.306 and the NMI 0.495. Table 3 shows what happened: k-Means grouped the beans mainly by size. One cluster is almost entirely Bombay (very large), one holds the small, rounder beans (Dermason, Seker and most Sira) and one holds the medium, longer beans (Horoz, Cali, Barbunya). Each bean type mostly stays together, but each cluster mixes several types. Out of interest I also checked $k = 7$ (the number of bean types) without using it for the choice. It gave a much better ARI (0.653) but a worse silhouette (0.303), because the seven types touch each other without clear gaps.

I noticed several limitations of k-Means here. First, k has to be chosen in advance, and the measures disagreed (2, 3 or 5); a stability check or the gap statistic could help. Second, k-Means expects round clusters of similar size, but the bean groups are stretched, overlapping and very different in size (82 Bombay against more than 1,000 small beans); a Gaussian mixture model would handle this better. Third, about eight of the features all measure size, so size is effectively counted several times and dominates the distances, which is why the clusters follow size; feature selection or PCA (Task 3.3B) could reduce this. Finally, the result depends on the starting points (runs with a single start only agree with each other at ARI 0.84), so I used 10 restarts.

3.2 DBSCAN

For MinPts I used the common rule of thumb of twice the number of features, $2 \times 16 = 32$, since a larger value gives steadier density estimates in 16 dimensions. To choose eps I sorted every bean's distance to its 32nd nearest neighbour (Figure 7a) and took the knee of the curve, where the distances suddenly shoot up. That gave **eps = 2.48**. Beans past the knee are in sparse areas and should count as noise. I also tried a grid of other values (Figure 7b). With MinPts = 32, every eps from 2.0 to 2.75 gave the same two clusters with 1–3% noise, so the choice is not fragile. Much smaller eps broke the data into pieces and labelled 8–53% of the beans as noise, while eps = 3.0 merged almost everything into one cluster.

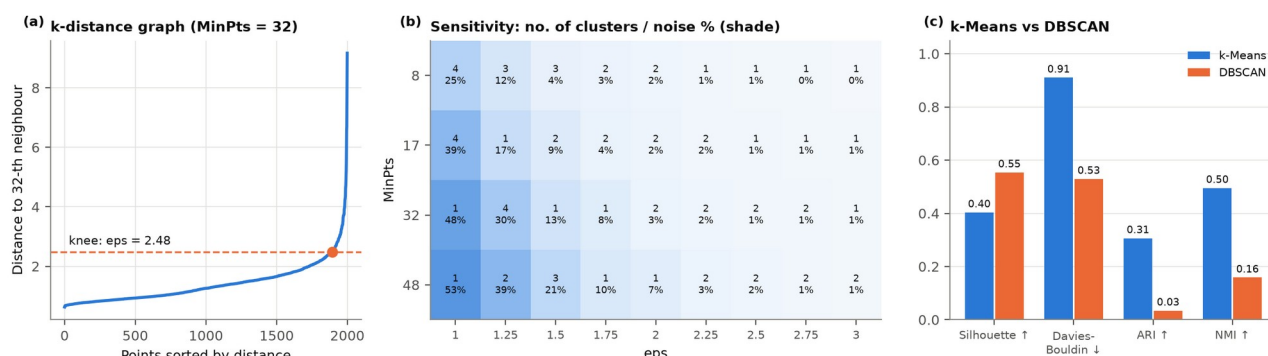
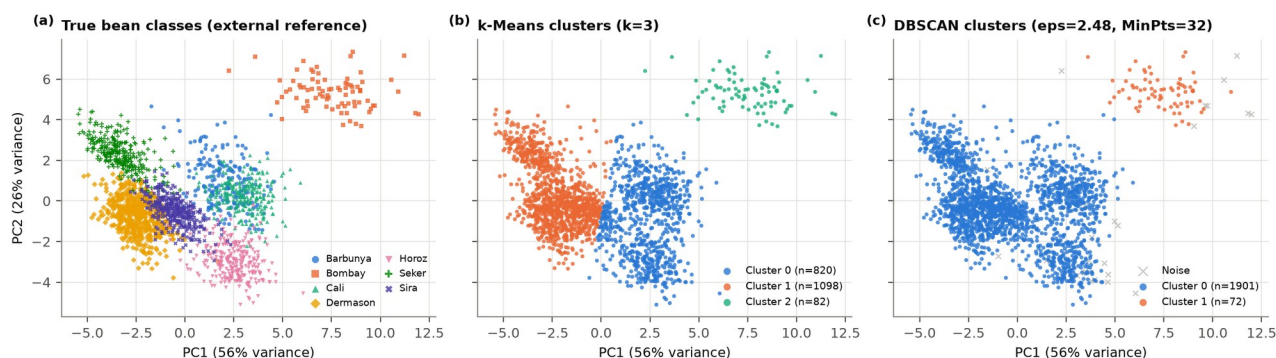
DBSCAN found **2 clusters** (1,901 and 72 beans) and **27 noise points (1.4%)**. The small cluster is made up only of Bombay beans, and the big one contains all six other types. Most noise points were unusually long Horoz beans (12) and Bombay beans (8).

Table 4. *k*-Means and DBSCAN on the same standardised data (DB = Davies–Bouldin, CH = Calinski–Harabasz; DBSCAN noise points are left out of the internal scores).

Algorithm	Clusters	Noise	Silhouette $\uparrow$	DB $\downarrow$	CH $\uparrow$	ARI $\uparrow$	NMI $\uparrow$
k-Means (k = 3)	3	0%	0.403	0.910	1,288	0.306	0.495
DBSCAN (eps = 2.48, MinPts = 32)	2	1.4%	0.554	0.529	557	0.034	0.158

The comparison (Table 4, Figure 7c) is interesting because the two kinds of measure disagree. DBSCAN has a better silhouette (0.554 vs 0.403) and Davies–Bouldin (0.529 vs 0.910), but it matches the real bean types far worse (ARI 0.034 vs 0.306). To check that the noise removal was not causing this, I scored k-Means on the same non-noise beans and got almost the same silhouette (0.408).

I think the main reason is how each algorithm defines a cluster. k-Means simply divides the space into *k* regions around centres, so it is happy to cut a continuous mass of beans into 'small' and 'medium' (Figure 8b). DBSCAN only separates regions with an empty, low-density gap between them. Apart from Bombay, the six bean types form one continuous cloud (Figure 8a), so DBSCAN chains them all together. Only Bombay is separated by a real gap. The 'Bombay vs everything else' split is very clean, which is why the internal scores like it, but it says little about the bean types. This showed me that internal scores alone can be misleading. DBSCAN's advantages are that it finds the number of clusters itself and flags outliers. Its weaknesses are that it uses one density level for everything (the small beans are packed tightly, Bombay is spread out) and that eps is hard to set in 16 dimensions. A variable-density method such as HDBSCAN might do better.

**Figure 7.** (a) Distance to the 32nd nearest neighbour, with the chosen eps at the knee; (b) number of clusters and noise % for other eps and MinPts values; (c) k-Means and DBSCAN on the same measures.**Figure 8.** The beans plotted on the first two principal components (for display only): (a) real classes, (b) k-Means clusters, (c) DBSCAN clusters (grey crosses are noise).

3.3B PCA Analysis

I applied PCA to the standardised features; without scaling, the first component would just be Area. The first component explains 55.6% of the variance, the second 26.0%, the third 8.1%, the fourth 5.2% and the fifth 2.9%. Together the first four explain 94.9% (Figure 9a). Components 9 to 16 explain almost nothing, because many of the features are calculated from each other (for example, EquivDiameter comes straight from Area).

To decide how many components to keep, I looked at several rules. The 'eigenvalue above 1' rule and the bend in the scree plot both suggest 3, keeping at least 90% of the variance suggests 4, and 95% suggests 5. I kept **components (94.9%)**. The third component alone only reaches 89.7%, and the fourth captures something none of the others do (Extent), while a fifth would add less than 3%.

The loadings (Figure 9b) make the components easy to interpret. PC1 is mainly size: all the size features load positively (MajorAxisLength contributes the most, 10.6%) and the roundness-type features negatively, so it

separates big, long beans from small, round ones. PC2 is shape regardless of size: width and compactness against aspect ratio (each about 11%). PC3 is how regular the outline is (Solidity 55%, ShapeFactor4 26%), and PC4 is almost entirely Extent (88%).

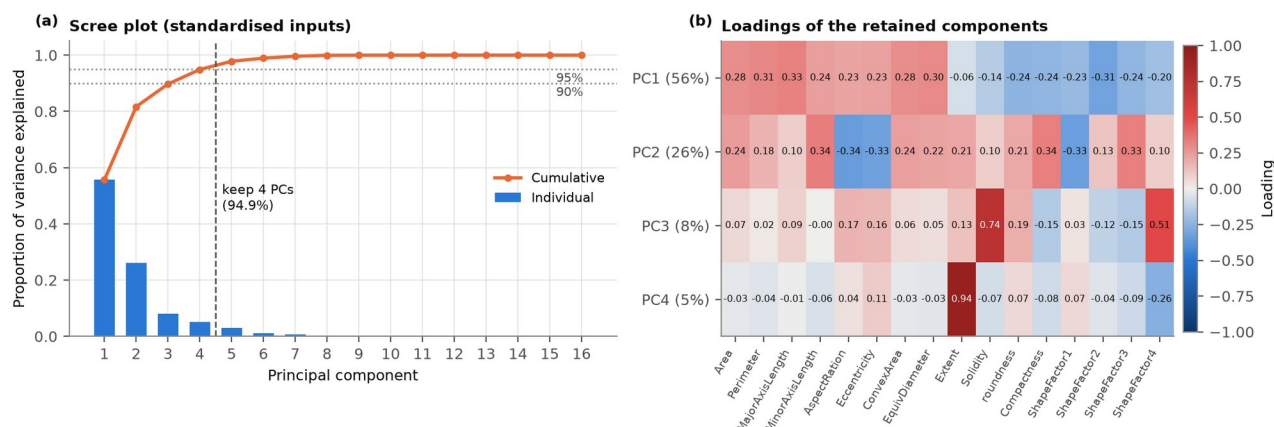


Figure 9. (a) Variance explained by each component and in total (dotted lines at 90% and 95%); (b) loadings of the 16 features on the four kept components.

I then ran k-Means on the 4 components. I rechecked the choice of k in this space and it still came out as 3, so I kept $k = 3$ to make the comparison fair. As an extra test I also tried 'whitened' components, where each component is rescaled to the same variance. Table 5 compares the three versions. Bootstrap stability is how closely the clusters from 80% subsamples match the full result, and seed stability is how much single-start runs agree with each other.

Table 5. *k*-Means ($k = 3$) on the original features compared with the PCA versions.

Representation	Silh. (own)	Silh. (orig.)	DB	ARI	NMI	Agree. *	Boot. stab.	Seed stab.	Cluster sizes
Original (16 variables)	0.403	0.403	0.910	0.306	0.495	1.000	0.997	0.841	1098/820/82
PCA (4 PCs)	0.421	0.403	0.862	0.304	0.492	0.990	0.995	0.809	1099/819/82
Whitened PCA (4 PCs)	0.241	0.246	1.516	0.243	0.325	0.531	0.699	0.360	960/554/486

* Agree. = how closely the clusters match the clusters from the original features (ARI). Silh. (orig.) is the silhouette of the same clusters measured on the original features.

With 4 components, k-Means found practically the same clusters as before: only 5 of the 2,000 beans changed cluster (agreement 0.990). The match with the real types (ARI 0.304 vs 0.306) and the stability (0.995 vs 0.997) barely changed, and the cluster sizes and their size-based meaning stayed the same. It did run about 2.5 times faster. At first the silhouette looked better in the PCA space (0.421 vs 0.403), but Figure 10a shows this is misleading. The fewer components I kept, the higher the silhouette in the reduced space (0.65 with one component), yet the same clusters measured on the original features stay at 0.40. Dropping dimensions makes the clusters look tighter without actually changing them.

I think there are a few reasons why PCA neither helped nor hurt here. k-Means works on distances, and PCA is basically a rotation, which does not change distances. Keeping six or more components gave exactly the same clusters, and dropping down to four only removed 5% of the variance. The size structure that k-Means finds lives in the first two components, which are always kept. PCA also removes the correlation between features, but it does not reduce the weight of size: the many correlated size features simply become PC1, which still carries 56% of the variance, so the clusters still follow size. The whitened version shows the other side of this. Giving every component equal weight boosted outline regularity and Extent, which carry mostly noise here. The clusters changed completely (agreement 0.53), the match with the real types dropped (ARI 0.24) and the results became unstable. So in this data the high-variance directions are where the useful structure is. Reducing too far does lose information: with only one component the clusters started to drift (agreement 0.94). I also tried DBSCAN on the 4 components, and it again only separated Bombay from the rest, which suggests the overlap between bean types is real and not just an effect of having many dimensions.

Overall, PCA gave a smaller, easier-to-explain version of the data with the same clustering quality and a faster run time, but not better clusters. Getting closer to the seven bean types would need a different way of weighting the features or a different cluster model, not just fewer dimensions.

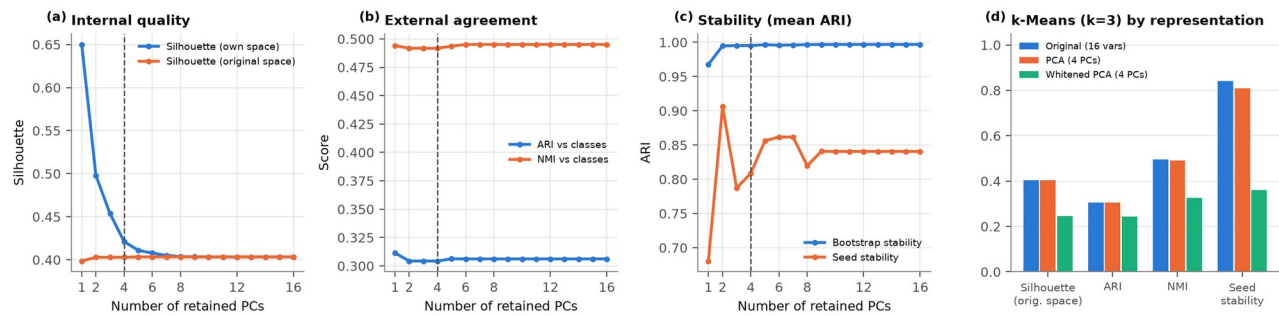


Figure 10. *k*-Means ($k = 3$) as the number of kept components changes: (a) silhouette in the reduced and original space, (b) match with the real classes, (c) stability; (d) the three versions side by side.

Acknowledgement of AI use

I used Claude (Anthropic, 2026) to help plan the analysis, write and check parts of the Python code, and improve the wording of this report. I ran all the code myself, and every number and figure in this report comes from my notebook outputs.

References

- Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai/>
- Gou, J., Zhan, Y., Rao, Y., Shen, X., Wang, X., & He, W. (2014). Improved pseudo nearest neighbor classification. *Knowledge-Based Systems*, 70, 361–375. <https://doi.org/10.1016/j.knosys.2014.07.020>
- Mitani, Y., & Hamamoto, Y. (2006). A local mean-based nonparametric classifier. *Pattern Recognition Letters*, 27(10), 1151–1159. <https://doi.org/10.1016/j.patrec.2005.12.016>